



中国石油大学(北京)

计算机科学与技术 专业

课程设计报告

课程设计名称：编译原理课程设计

学 院 人工智能学院

专 业 计算机

班 级 23-3

姓 名 郭宇杰

指导教师 朱雪峰

编译原理课程设计报告

目录

实践一 词法分析与语法分析.....	2
编译原理课程设计报告：实践 1.1 Flex 词法分析.....	2
一、目的和任务.....	2
二、过程.....	2
三、结果与结论分析.....	5
编译原理课程设计报告：实践 1.2 Bison 语法分析.....	6
一、目的和任务.....	6
二、过程.....	7
三、结果与结论分析.....	8
实践二 语义分析.....	10
编译原理课程设计报告：实践 2 语义分析.....	10
一、目的和任务.....	10
二、过程.....	11
三、结果与结论分析.....	12
实践三 中间代码生成.....	14
编译原理课程设计报告：实践 3 中间代码生成.....	14
一、目的和任务.....	15
二、过程.....	15
三、结果与结论分析.....	17
实践四 目标代码生成.....	19
编译原理课程设计报告：实践 4 目标代码生成.....	19
一、目的和任务.....	20
二、过程.....	20
三、结果与结论分析.....	21
实践五 中间代码优化.....	23
编译原理课程设计报告：实践 5 中间代码优化.....	23
一、目的和任务.....	23
二、过程.....	23
三、结果与结论分析.....	25
工作总结.....	26

实践一 词法分析与语法分析

编译原理课程设计报告：实践 1.1 Flex 词法分析

实践内容：实践 1.1 学习 Flex，用 Flex 实现词法分析

一、目的和任务

本次实践的目标是根据 C++ 语言词法规则，使用 GNU Flex 实现一个词法分析程序。词法分析程序需要读取 C++ 源代码文件，将字符流识别为词法单元，并对不符合词法规则的输入给出错误类型、错误行号和说明文字。

本阶段主要任务如下：

1. 阅读课程资料中附录 A 的 C++ 语言文法，整理需要识别的 token 类型。
2. 使用 Flex 编写词法规则，识别关键字、标识符、整数、浮点数、运算符、分隔符等词法单元。
3. 过滤空白字符和注释，支持 `//` 单行注释和 `/* ... */` 多行注释。
4. 对未定义字符、非法标识符、非法整数、非法浮点数、非法注释等情况输出 A 类词法错误。
5. 编写构建和测试命令，保证词法分析程序可以重复编译、重复验证。

二、过程

1. 文法和 token 整理

根据 `Appendix.pdf`，C++ 语言需要识别的主要词法单元包括：

- 基本字面量：`INT`、`FLOAT`、`ID`
- 关键字：`int`、`float`、`struct`、`return`、`if`、`else`、`while`
- 运算符：`=`, 关系运算符、`+`、`-`、`\*`、`/`、`&&`、`||`、`.`、`!`
- 分隔符：`;`、`,`、`(`、`)`、`[`、`]`、`{`、`}`

为便于后续语法分析复用，本次实现将 token 类型统一定义在 `src/token.h` 中，Flex 规则返回这些枚举值。独立的词法分析入口位于 `src/lexer\_main.c`。

2. 正则表达式设计

词法规则写在 `src/lexer.l` 中。主要正则定义如下：

DIGIT	[0-9]
NZDIGIT	[1-9]
LETTER	[_a-zA-Z]
IDCHAR	[_a-zA-Z0-9]
DEC_INT	0 {NZDIGIT}{DIGIT}*
OCT_INT	0[0-7]+
HEX_INT	0[xX][0-9a-fA-F]+
EXP	[eE][+-]?{DIGIT}+

```

FLOAT_BASE  ({DIGIT}+\.{DIGIT}+|{DIGIT}+\.\.{DIGIT}+)
FLOAT_LIT   ({FLOAT_BASE}{EXP}?|{DIGIT}+{EXP})
ID          {LETTER}{IDCHAR}*

```

其中整数支持十进制、八进制和十六进制。浮点数支持普通小数和指数形式，例如 `9.1e-2`、`2.99e8`、`1.0E+3`。标识符要求以字母或下划线开头，后续可以包含字母、数字和下划线。

3. 注释和空白处理

空格、制表符、换行等空白字符不返回 token。单行注释使用规则 `"/"[\n]\*` 直接过滤。

多行注释使用 Flex 的 start condition 实现：

```
%x COMMENT
```

读到 `/\*` 后进入 `COMMENT` 状态，读到 `\*/` 后回到初始状态。若注释中再次遇到 `/\*`，输出嵌套注释错误；若到文件结束仍没有遇到 `\*/`，输出未闭合注释错误。

4. 错误处理

实践要求中词法错误属于错误类型 A，输出格式为：

Error type A at Line [行号]: [说明文字].

本次实现中，词法错误通过 `lexical\_error\_count` 计数。默认运行时只输出词法错误，不输出 token 流；这样可以避免在错误用例中混入调试信息。需要调试 token 流时，可设置环境变量 `CMM\_LEX\_DUMP=1`。

已实现的 A 类错误包括：

- 未定义字符，例如 `\$`
- 数字开头的非法标识符，例如 `2x`
- 非法浮点数，例如 `3.14.15`
- 非法十六进制数，例如 `0xQQQ`
- 非法八进制数，例如 `099`
- 非法指数浮点数，例如 `2.5e`、`1.0E+`
- 嵌套块注释
- 未闭合块注释

5. 构建和测试

项目中添加了统一的 Makefile 入口：

```
make lexer
```

该命令会执行：

```
flex -o build/lexer.c src/lexer.l  
gcc -std=c11 -Wall -Wextra -D_POSIX_C_SOURCE=200809L -I src  
src/lexer_main.c build/lexer.c -lfl -o build/cmm-lexer
```

词法测试命令为：

```
make lexer-test
```

测试脚本位于 `tools/run\_lexer\_tests.sh`，会使用文件夹 1 中的测试样例进行回归测试。

三、结果与结论分析

1. 测试结果

执行：

```
make lexer-test
```

输出结果为：

```
ok: A-1
ok: A-2
ok: A-5
ok: E1-2
ok: E2-2
ok: E3-2
lexer tests: 6 passed, 0 failed
```

其中:

- `A-1` 检查未定义字符 `\$`
- `A-2` 检查数字开头的非法标识符 `2x`
- `A-5` 检查非法浮点常量 `3.14.15`
- `E1-2` 检查非法十六进制和八进制常量
- `E2-2` 检查非法指数浮点常量
- `E3-2` 检查嵌套块注释

说明词法分析器已经能够识别实践 1.1 所需的主要 token，并能按要求报告 A 类词法错误。

2. 问题和解决

实现过程中遇到的主要问题是十六进制整数与“数字开头的非法标识符”规则之间存在匹配冲突。例如 `0xFFFF` 原本可能被错误规则识别为非法标识符。解决方法是调整 Flex 规则顺

序，并精确限制非法十六进制规则，只在 `0x` 后出现非十六进制字符时报告错误。

另一个问题是测试期望文件中部分文件没有末尾换行，直接使用 `diff` 比较会出现内容相同但测试失败的情况。测试脚本中对比较文本做了规范化处理，避免末尾换行差异影响本地回归结果。

3. 结论

本次实践完成了基于 Flex 的 C-- 词法分析器，实现了 token 识别、注释过滤、行号维护和 A 类错误报告。当前代码保留了清晰的 token 枚举和词法入口，后续实践 1.2 可以在此基础上接入 Bison，将词法分析器作为语法分析器的输入模块继续扩展。

编译原理课程设计报告：实践 1.2 Bison 语法分析

实践内容：实践 1.2 学习 Bison，用 Bison 实现语法分析

一、目的和任务

本次实践的目标是在实践 1.1 词法分析器的基础上，使用 GNU Bison 实现 C-- 语言的语法分析程序。程序需要判断输入源代码是否符合 C-- 语法规则；若无词法和语法错误，则按照课程要求输出语法树；若存在错误，则输出错误类型、行号和说明文字。

主要任务如下：

1. 根据附录 A 的 C-- 文法编写 Bison 产生式。
2. 设置表达式运算符优先级和结合性，解决赋值、逻辑、关系、算术、数组访问和结构体访问等表达式的二义性。
3. 构造语法树，并按先序遍历格式输出。
4. 复用 Flex 词法分析器，将 token 和属性值传递给 Bison。
5. 对常见语法错误进行恢复，尽量避免一个错误导致大量级联报错。
6. 使用文件夹 1 的测试样例进行回归验证。

二、过程

1. 词法分析器和语法分析器衔接

实践 1.1 中的 ``src/lexer.l`` 被继续复用。为了同时支持独立词法测试和 Bison 语法分析，本次实现增加了 ``PARSER_BUILD`` 编译开关：

- 独立 lexer 模式下，Flex 返回 ``token.h`` 中定义的枚举。
- parser 模式下，Flex 引入 ``parser.tab.h``，返回 Bison token，并通过 ``yylval`` 传递语法树终结符节点。

这样可以避免维护两套词法规则，保证词法识别行为一致。

2. 语法树数据结构

新增 `src/ast.h` 和 `src/ast.c` 作为语法树模块。每个节点包含：

- 节点类型：语法单元或词法单元
- 节点名称
- 可选属性值
- 行号
- 子节点列表

非终结符节点打印格式为：

`NodeName (line)`

终结符节点按课程要求打印。其中 `ID`、`TYPE`、`INT`、`FLOAT` 需要额外输出属性值，`INT` 使用十进制输出，`FLOAT` 使用 `%f` 输出。

3. Bison 文法实现

语法规则写在 `src/parser.y` 中，覆盖附录 A 中的主要产生式：

- `Program`、`ExtDefList`、`ExtDef`

- ``Specifier``、``StructSpecifier``
- ``VarDec``、``FunDec``、``VarList``
- ``CompSt``、``StmtList``、``Stmt``
- ``DefList``、``Def``、``DecList``、``Dec``
- ``Exp``、``Args``

表达式优先级按附录 A 表格设置：

赋值 < || < && < 关系运算 < + - < * / < 一元 - 和 ! < 数组/结构体/函数调用

同时使用 ``LOWER_THAN_ELSE`` 和 ``ELSE`` 解决 dangling else 问题。

4. 错误恢复

除了 Bison 默认语法错误外，还根据文件夹 1 的测试要求增加了若干可恢复错误产生式，例如：

- 数组维度不是整数
- 函数形参列表尾逗号
- 函数调用实参列表尾逗号
- 全局变量初始化

- 局部变量定义出现在语句之后
- 空 while 语句体
- 结构体匿名声明位置错误
- 数组初始化列表不支持
- 结构体成员访问缺少成员名

这些错误被报告为 B 类语法错误。报告后构造占位语法节点继续分析，减少级联错误。

三、结果与结论分析

1. 测试结果

执行：

```
make parser-test
```

输出：

```
ok: A-1  
ok: A-10  
ok: A-2  
ok: A-3  
ok: A-4  
ok: A-5
```

```
ok: A-6
ok: A-7
ok: A-8
ok: A-9
ok: B-1
ok: B-2
ok: C-1
ok: C-2
ok: D-1
ok: D-2
ok: D-3
ok: E1-1
ok: E2-1
ok: E3-1
parser tests: 20 passed, 0 failed
```

测试覆盖了词法错误、语法错误、多个错误恢复、结构体、数组、函数、循环、表达式优先级、八进制和十六进制整数、指数浮点数以及注释过滤。

2. 问题和解决

实现过程中主要问题是错误恢复。如果只使用 Bison 默认错误处理，一个语法错误容易导致后续多行被误报。为解决该问题，在 `VarDec`、`FunDec`、`Stmt`、`Def`、`Dec`、`Args` 等局部位置加入了恢复产生式，使错误能在较小范围内被消费，并继续分析后续代码。

另一个问题是语法树输出格式必须严格匹配课程样例。为此，语法树模块将空产生式节点直接省略，并统一使用子节点首行作为非终结符行号。整数和浮点数在打印时转换为十进制格式，保证与期望输出一致。

3. 结论

本次实践完成了基于 Bison 的 C-- 语法分析器，并与 Flex 词法分析器完成衔接。程序能够在无错误时输出符合课程格式的语法树，在有错误时输出 A 类或 B 类错误信息。文件夹 1 中全部 20 个期望输出均已通过，说明实践 1.2 的核心功能已经完成，可作为后续语义分析实践的基础。

实践二 语义分析

编译原理课程设计报告：实践 2 语义分析

实践内容：实践 2 语义分析

一、目的和任务

本次实践的目标是在实践 1 词法分析和语法分析的基础上，实现 C-- 语言的语义分析程序。

程序需要在语法分析成功后检查变量、函数、结构体和表达式的语义约束；若发现语义错误，则按照课程要求输出错误类型、行号和说明文字；若没有语义错误，则不输出内容。

主要任务如下：

1. 建立 C-- 语言的类型系统，支持 `int`、`float`、数组、结构体和函数类型。

2. 建立符号表，记录变量、函数、结构体定义和作用域信息。
3. 检查未定义变量、未定义函数、变量和函数重定义、结构体重重复定义、未定义结构体等错误。
4. 检查赋值、运算、返回语句、数组访问、结构体成员访问和函数调用参数的类型一致性。
5. 支持函数声明和定义，并检查未定义函数声明与声明不一致问题。
6. 使用 Project 2 的 30 个测试样例进行回归验证。

二、过程

1. 语义分析入口

语法分析仍然由 ``src/parser.y`` 和 ``src/lexer.l`` 完成。为了不破坏实践 1.2 的语法树输出，``src/parser_main.c`` 保留默认行为：语法正确时打印语法树。实践 2 通过环境变量开启语义分析：

```
CMM_SEMANTIC=1 build/cmmc tests/project2/inputs/A-1.cmm
```

当存在词法错误或语法错误时，程序不进入语义分析，直接输出已有的 A/B 类错误。只有语法分析成功后，才调用 ``semantic_check(parse_root)`` 遍历语法树。

2. 类型系统

新增 `src/semantic.c` 和 `src/semantic.h`。类型系统使用 `Type` 表示：

- 基本类型：`int`、`float`
- 数组类型：记录元素类型和维度大小
- 结构体类型：记录结构体名和字段链表
- 函数类型：记录返回类型、参数类型列表、参数个数和是否已定义

类型比较时，基本类型要求种类相同；数组要求维度和元素类型相同；具名结构体按结构体名比较，匿名结构体按字段类型序列比较。

3. 符号表和作用域

符号表使用链表实现，并通过 `Scope` 维护嵌套作用域。全局作用域记录全局变量，函数表单独记录函数声明和定义，结构体表记录结构体定义。

变量声明时检查同一作用域内是否重定义；函数定义时检查是否重复定义；结构体定义时检查结构体名是否重复。进入复合语句时创建新作用域，退出时恢复外层作用域。

4. 语义检查规则

语义分析按 AST 递归遍历：

- `ExtDef`：处理全局变量、结构体定义、函数声明和函数定义。

- ``DefList``、``DecList``：处理局部变量和结构体字段声明。
- ``Stmt``：检查返回语句、条件语句、循环语句和表达式语句。
- ``Exp``：综合表达式类型，并检查赋值、二元运算、函数调用、数组访问和结构体成员访问。

实现覆盖了 Project 2 样例中的 1-19 类错误，包括：

- 未定义变量、未定义函数
- 变量、函数、字段、结构体重定义
- 赋值类型不匹配和左值错误
- 运算对象类型不匹配
- 返回类型不匹配
- 函数调用实参与形参不匹配
- 非数组访问、数组下标非整数
- 非结构体成员访问、不存在字段
- 函数声明未定义和声明不一致

5. 测试脚本

新增 ``tools/run_semantic_tests.sh``，对 ``tests/project2/expect`` 下所有 ``.exp`` 文件逐个运行：

```
CMM_SEMANTIC=1 build/cmmc tests/project2/inputs/<case>.cmm
```

并使用 `diff -u` 与期望输出逐字比较。`Makefile` 增加：

```
make semantic  
make semantic-test
```

三、结果与结论分析

1. 测试结果

执行：

```
make semantic-test
```

输出：

```
ok: A-1  
ok: A-10  
ok: A-11  
ok: A-12  
ok: A-13  
ok: A-14  
ok: A-15  
ok: A-16  
ok: A-17
```

```
ok: A-18
ok: A-19
ok: A-2
ok: A-20
ok: A-3
ok: A-4
ok: A-5
ok: A-6
ok: A-7
ok: A-8
ok: A-9
ok: B-1
ok: B-2
ok: C-1
ok: C-2
ok: D-1
ok: D-2
ok: D-3
ok: E-1
ok: E-2
ok: E-3
semantic tests: 30 passed, 0 failed
```

同时回归实践 1:

```
lexer tests: 6 passed, 0 failed
parser tests: 20 passed, 0 failed
```

说明新增语义分析没有破坏实践 1.1 和 1.2 的输出。

2. 问题和解决

实现过程中主要问题是语义错误不能产生级联误报。例如数组访问对象不是数组时，后续赋值检查不能再额外报告左值错误。为此，错误类型在表达式分析中会返回 `TY\_ERROR`，后续类型比较遇到错误类型时不再重复报告。

另一个问题是函数声明和定义的关系。实现中函数表单独保存函数类型，声明和定义通过返回类型、参数个数和参数类型列表判断是否一致。语义分析结束后统一检查只有声明没有定义的函数。

Project 2 的 D-2 和 E-2 样例分别对应不同作用域假设：D-2 按课程旧假设检查外层变量重名 E-2 按要求 3.2 支持嵌套作用域。实现默认使用嵌套作用域；测试脚本只在运行 D-2 时通过 `CMM\_NO\_SCOPE=1` 显式开启旧口径兼容模式。

3. 结论

本次实践完成了 C++ 语言的语义分析器。程序能够在语法分析成功后检查变量、函数、数组、结构体和表达式相关语义错误，并按课程样例输出错误信息。Project 2 的 30 个测试样例全部通过，实践 2 的核心功能已经完成，可作为后续中间代码生成实践的基础。

实践三 中间代码生成

编译原理课程设计报告：实践 3 中间代码生成

实践内容：实践 3 中间代码生成

一、目的和任务

本次实践的目标是在词法分析、语法分析和语义分析完成的基础上，将 C-- 源程序翻译为线性中间代码。中间代码采用三地址代码形式，用于描述赋值、算术运算、条件跳转、函数调用、参数传递、数组访问、结构体字段访问和输入输出操作。

主要任务如下：

1. 在语义检查通过后遍历 AST，生成中间代码文件。
2. 生成临时变量和跳转标号，翻译表达式、语句和控制流。
3. 翻译函数定义、形参、实参、函数调用和返回语句。
4. 翻译局部数组和结构体变量的空间申请、地址计算和间接读写。
5. 特殊处理内置函数 ``read`` 和 ``write``，生成 ``READ``、``WRITE`` 指令。
6. 对 Project 3 的样例输入批量生成 IR 并进行验证。

二、过程

1. IR 生成入口

为了不破坏前两个实践的运行方式，程序默认仍保持语法树输出；实践 2 使用

`CMM\_SEMANTIC=1` 进入语义分析模式；实践 3 使用 `CMM\_IR=1` 进入 IR 生成模式，并额外接收输出文件路径：

```
CMM_IR=1 build/cmmc tests/project3/inputs/A-1.cmm build/project3-ir/A-1.ir
```

IR 模式下，程序先执行词法、语法和语义检查。只有输入无错误时才调用 `ir\_generate(parse\_root, output\_path)` 输出中间代码。

2. 类型和符号信息

新增 `src/irgen.c` 和 `src/irgen.h`。IR 生成阶段单独维护一套轻量级类型与符号信息：

- `IrType`：表示 `int`、`float`、数组和结构体。
- `IrField`：表示结构体字段或函数参数，并记录字段偏移。
- `IrSymbol`：表示当前作用域中的变量，记录变量类型以及是否为地址参数。
- `IrFunc`：表示函数签名，用于判断实参传值还是传地址。

结构体字段偏移和数组元素宽度通过类型大小递归计算。基本类型大小为 4 字节，数组大小为元素大小乘以长度，结构体大小为所有字段大小之和。

3. 表达式和语句翻译

表达式翻译返回一个可用于 IR 的地址字符串，例如变量名、临时变量或立即数。普通算术

表达式生成：

```
t1 := x + y
```

赋值语句根据左值形式区分直接赋值和间接赋值：

```
x := t1
*t2 := t1
```

数组访问和结构体字段访问先生成地址，再通过 `\*addr` 读写内存：

```
t1 := i * #4
t2 := &a + t1
t3 := *t2
```

条件语句和循环语句通过 `LABEL`、`GOTO`、`IF ... GOTO` 翻译。布尔表达式在控制流上

下文中直接生成跳转代码，在普通表达式上下文中转换为 `0` 或 `1`。

4. 函数和参数翻译

函数定义输出：

```
FUNCTION f :  
PARAM x
```

普通整型参数按值传递；数组和结构体参数按地址传递。调用函数时先逆序输出 `ARG`，再输出 `CALL`：

```
ARG y  
ARG x  
t1 := CALL f
```

内置函数特殊处理：

```
READ t1  
WRITE x
```

5. 测试脚本

新增 `tools/run\_ir\_tests.sh`，对 Project 3 的 16 个样例逐个生成 `.ir` 文件，并检查输出非空且包含 `FUNCTION` 指令。生成目录为：

```
build/project3-ir/
```

三、结果与结论分析

1. 测试结果

执行：

```
make ir-test
```

输出：

```
ok: A-1 -> build/project3-ir/A-1.ir
ok: A-2 -> build/project3-ir/A-2.ir
ok: A-3 -> build/project3-ir/A-3.ir
ok: A-4 -> build/project3-ir/A-4.ir
ok: A-5 -> build/project3-ir/A-5.ir
ok: B-1 -> build/project3-ir/B-1.ir
ok: B-2 -> build/project3-ir/B-2.ir
ok: B-3 -> build/project3-ir/B-3.ir
ok: C-1 -> build/project3-ir/C-1.ir
ok: C-2 -> build/project3-ir/C-2.ir
ok: E1-1 -> build/project3-ir/E1-1.ir
ok: E1-2 -> build/project3-ir/E1-2.ir
ok: E1-3 -> build/project3-ir/E1-3.ir
ok: E2-1 -> build/project3-ir/E2-1.ir
ok: E2-2 -> build/project3-ir/E2-2.ir
ok: E2-3 -> build/project3-ir/E2-3.ir
ir tests: 16 generated, 0 failed
```

同时回归前两部分：

```
lexer tests: 6 passed, 0 failed
parser tests: 20 passed, 0 failed
semantic tests: 30 passed, 0 failed
```

课程提供的 `irsim.pyc` 是 Python 3.8 字节码，并依赖 PyQt5。当前通过 `tools/build\_irsim\_image.sh` 构建 `cmm-irsim-py38` 镜像，提供 Python 3.8、PyQt5 和 Qt 运行依赖。测试脚本通过截取 `irsim.pyc` 中的类定义部分，绕过 GUI 主事件循环，直接调用 `IRSim.loadFile()` 和 `IRSim.run()`，并用 monkeypatch 为 `READ` 指令提供测试输入。

执行：

```
make irsim-test
```

输出：

```
irsim tests: 16 passed, 0 failed
```

2. 问题和解决

实现中的重点是左值地址和右值求值的区分。变量赋值可以直接生成 `x := y`，但数组元素

和结构体字段需要先计算地址，再生成 `*addr := y` 或 `x := *addr`。为了避免生成 `*&x` 这类不稳定格式，若地址表达式是 `&x`，会先将其保存到临时变量再进行间接访问。

另一个问题是函数参数传递。普通整型参数直接传值，数组和结构体参数传地址。IR 生成阶段会先收集函数签名，在翻译函数调用时根据形参类型决定实参使用值还是地址。

3. 结论

本次实践完成了 C-- 到三地址中间代码的生成。程序支持 Project 3 样例中的必做内容，并扩展支持结构体变量、结构体参数、高维数组和一维数组参数。生成的中间代码格式符合课程说明，可作为后续目标代码生成和优化实践的输入。

实践四 目标代码生成

编译原理课程设计报告：实践 4 目标代码生成

实践内容：实践 4 目标代码生成

一、目的和任务

本次实践的目标是在已有词法分析、语法分析、语义分析和中间代码生成的基础上，将 C-- 程序翻译为可在 SPIM Simulator 中运行的 MIPS32 汇编代码。

主要任务如下：

1. 增加目标代码生成入口，接收 C-- 输入文件和 `.s` 输出文件。
2. 复用实践 3 的线性 IR，完成从 IR 到 MIPS32 指令的翻译。
3. 实现函数栈帧、参数传递、返回值、递归调用和局部数据存储。
4. 翻译赋值、算术运算、条件跳转、循环、数组地址计算和间接访存。
5. 使用 SPIM 对生成的汇编程序进行运行验证。

二、过程

1. 目标代码生成入口

程序默认仍保持语法树输出；`CMM\_SEMANTIC=1` 进入语义分析模式；`CMM\_IR=1` 进入中间代码生成模式。本次实践新增 `CMM\_MIPS=1`：

```
CMM_MIPS=1          build/cmmc          tests/project3/inputs/A-5.cmm  
build/project4-mips/A-5.s
```

为了贴合实践四的提交要求，当前也支持直接使用两个参数生成汇编：

```
build/cmmc tests/project3/inputs/A-5.cmm build/project4-mips/A-5.s
```

该模式下，程序先完成词法、语法和语义检查。检查通过后，先生成临时 IR 文件，再调用 MIPS 后端输出汇编文件。这样可以复用实践 3 的中间表示，也便于单独调试 IR 和目标代码两阶段。一参数运行时仍保留实践 1.2 的语法树输出。

2. IR 到 MIPS 的翻译

新增 `src/mips-gen.c` 和 `src/mips-gen.h`。后端先读取文本 IR，把指令按函数分组，并收集每个函数中的变量、临时变量、参数和 `DEC` 空间。之后为每个函数计算栈帧布局。

当前实现采用朴素内存分配策略：所有变量和临时变量都存放在栈帧中，执行每条 IR 指令时从内存加载操作数到 `$t0`、`$t1` 等寄存器，计算后再写回栈帧。这种策略生成的代码不追求寄存器利用率，但结构简单，能自然支持递归和函数嵌套调用。

3. 栈帧和函数调用

每个函数入口建立独立活动记录，保存 `$ra` 和旧 `$fp`。局部变量、临时变量、数组和结构体展开后的存储空间都位于当前 `$fp` 的负偏移位置。

函数调用时，调用者按 IR 中 `ARG` 出现的顺序把实参值压栈，然后执行 `jal`。被调函数进入后，根据 `PARAM` 顺序从调用者压入的参数区读取参数，并复制到自己的局部槽位。函数返回后，调用者回收参数空间。由于每次调用都有独立栈帧，`fib` 这类递归函数可以正确运行。

4. 指令选择

主要 IR 指令的翻译方式如下：

- ``x := y``：加载 ``y``，写入 ``x`` 的栈槽。
- ``x := y + z``、``x := y - z``、``x := y * z``、``x := y / z``：加载两个操作数，使用 ``add``、``sub``、``mul``、``div/mflo``。
- ``x := &a``：计算 ``a`` 在当前栈帧中的地址。
- ``x := *p``：把 ``p`` 作为地址，使用 ``lw`` 读取。
- ``*p := x``：把 ``p`` 作为地址，使用 ``sw`` 写入。
- ``IF x relop y GOTO label``：使用 ``beq``、``bne``、``bgt``、``blt``、``bge``、``ble``。
- ``READ x``：使用 SPIM syscall 5。
- ``WRITE x``：使用 SPIM syscall 1 输出整数，并输出换行。

三、结果与结论分析

1. 测试结果

执行：

```
make mips-test
```

输出：

```
mips tests: 18 passed, 0 failed
```

测试覆盖了 Project 4 PDF 中的 2 个公开样例，以及 Project 3 的 16 个样例，包括整数变量、算术表达式、条件、循环、函数调用、递归、一维数组、地址传参、结构体字段展开和高维数组展开。虽然实践 4 文档的必做假设不要求结构体和高维数组变量，但由于实践 3 的 IR 已经把这些访问统一为地址计算和间接读写，当前 MIPS 后端也能通过这些样例。

完整回归执行：

```
make test
```

覆盖实践 1-4：词法测试、语法测试、语义测试、IRSim 运行验证和 SPIM 运行验证。

2. 问题和解决

实现中的关键问题是函数调用参数顺序。实践 3 的 IR 会逆序输出 `ARG`，例如调用 `pairVal(i, j)` 时会先输出 `ARG j`，再输出 `ARG i`。目标代码生成时按 IR 出现顺序压栈，使栈顶保存第一个形参对应的实参，被调函数再按 `PARAM` 顺序读取参数，从而恢复正确调用语义。

另一个重点是地址和值的区分。`&x` 表示栈帧中的地址，`\*p` 表示对地址间接读写。后端统一把数组、结构体字段和高维数组访问看成地址运算，因此不需要在 MIPS 阶段重新理解高级类型结构。

3. 结论

本次实践完成了 C-- 到 MIPS32 汇编的目标代码生成。程序能够生成可被 SPIM 执行的汇编代码，并通过已有样例的运行结果验证。至此，编译器已经形成从源程序到目标汇编的完整主流程。

实践五 中间代码优化

编译原理课程设计报告：实践 5 中间代码优化

实践内容：实践 5 中间代码优化

一、目的和任务

本次实践的目标是在实践三生成的中间代码基础上，对 IR 进行语义保持的优化，使输出代码更加简洁，并降低后续解释执行或目标代码生成的操作数量。

主要任务如下：

1. 实现独立 IR 优化器，输入`.ir`文件，输出优化后的`.ir`文件。
2. 完成常量传播、常量折叠、复制传播、局部公共子表达式消除和无用赋值删除。
3. 在优化过程中保持函数调用、控制流、数组和结构体地址访问的语义正确。
4. 使用 IRSim 验证优化后 IR 的运行结果与原程序一致。

二、过程

1. 优化器入口

新增`src/iropt.c`、`src/iropt.h`和`src/iropt\_main.c`，构建目标为：

```
make opt
```

运行方式为：

```
build/cmm-opt input.ir output.ir
```

该入口直接处理实践三格式的文本 IR，不依赖 AST 或语义分析结果，便于单独测试优化阶段。

2. 基本块内传播和折叠

优化器按行扫描 IR，在函数入口、标号、跳转、条件跳转和返回等边界处划分基本块。每个基本块内部维护变量到已知值的映射。对于 `x := #1`、`x := y` 这类赋值，记录常量或复制关系；后续指令中的操作数如果可替换，就直接改写为已知值。

当二元表达式的两个操作数都是常量时，优化器在编译期计算结果，例如：

```
t1 := #30 / #5
```

会优化为：

```
t1 := #6
```

3. 公共子表达式消除

在同一基本块内，优化器记录已经计算过的二元表达式。如果后续出现完全相同的表达式，且相关操作数和结果变量没有被重新定义，则复用原来的计算结果：

```
t3 := v1 + #2  
t7 := v1 + #2
```

可以优化为：

```
t3 := v1 + #2  
t7 := t3
```

对于加法和乘法，还会对两个操作数做规范化，使 `a + b` 和 `b + a` 能够被识别为同一个表达式。

4. 无用赋值删除

优化器会删除两类安全的无用赋值：

- 变量在同一基本块内被重新定义，且旧值在重新定义前没有被使用。
- 临时变量赋值后在整个函数后续代码中都没有被使用。

为了保证正确性，间接写 `\*p := x`、函数调用、`READ`、`WRITE`、`RETURN`、`ARG` 和控制流指令都不会被删除。

在局部删除之后，优化器进一步构造行级控制流后继关系，并进行反向活跃变量分析。对于每条赋值指令，如果左值变量不在该指令的 live-out 集合中，并且该指令没有副作用，则可以删除。由于数组和结构体访问可能通过地址间接读写，若变量出现过 `&x`，优化器会保守地保留对 `x` 的写入。

三、结果与结论分析

1. 测试结果

执行：

```
make opt-test
```

输出：

```
opt tests: 16 passed, 0 failed, saved 38 IR lines
```

测试脚本先为 Project 3 的 16 个样例生成原始 IR，再使用优化器生成优化后 IR，最后通过课程 IRSim 运行优化后 IR 并比较输出结果。所有样例输出一致，说明当前优化保持了程序语义。

此外，`examples/project5/` 中整理了实践五 PDF 的三个必做样例：局部公共子表达式消除、局部无用代码消除和常量折叠。`make opt-test` 会额外确认这三个公开样例都能被优化器缩减。

完整回归执行：

```
make test
```

覆盖实践 1-5：词法测试、语法测试、语义测试、原始 IRSim 验证、SPIM 验证和优化后 IRSim 验证。

2. 问题和解决

优化的主要风险是把局部成立的值传播到控制流边界之外。为避免错误，当前实现只在基本块内部保留传播状态，遇到函数入口、标号、跳转、条件跳转、返回和函数调用时清空状态。

另一个风险是数组和结构体访问引入的间接内存操作。当前实现不会删除间接写，也不会假设 `*p` 读取结果恒定，因此不会因为别名问题改变程序语义。

3. 结论

本次实践完成了一个保守的 IR 优化器，支持常量传播、常量折叠、复制传播、局部公共子表达式消除，以及基于活跃变量分析的无用赋值删除。优化后 IR 能通过课程 IRSim 验证，并可继续作为目标代码生成阶段的输入。

工作总结

本次课程设计从 C++ 源程序出发，逐步实现了词法分析、语法分析、语义分析、中间代码生成、目标代码生成和中间代码优化。项目最终形成了一个可以将 C++ 程序翻译为 MIPS32 汇

编并在 SPIM 中运行的简易编译器，同时提供了独立的 IR 优化器。

实践一完成了 Flex 与 Bison 的前端基础，能够识别 C++ token、过滤注释、报告词法错误，并在语法正确时输出课程要求格式的语法树。实践二在语法树基础上实现类型系统和符号表，支持变量、函数、数组、结构体和作用域相关的语义检查。实践三把语义正确的程序翻译为三地址形式中间代码，并通过课程 IRSim 验证执行结果。实践四在 IR 基础上生成 MIPS32 汇编，完成函数栈帧、参数传递、递归调用、条件跳转、数组/结构体地址访问和读写系统调用。实践五实现保守的 IR 优化，包括常量传播、常量折叠、复制传播、局部公共子表达式消除和无用赋值删除，并保持优化前后运行结果一致。

整个实现过程中，核心原则是保持各阶段接口清晰：词法分析输出 token，语法分析构造 AST，语义分析检查 AST 并维护符号信息，中间代码生成把高级结构降为统一的线性 IR，目标代码生成只依赖 IR 完成 MIPS 翻译，优化器直接处理文本 IR。这样既便于分阶段测试，也便于在后续阶段复用前面阶段的结果。

当前项目已经通过实践 1-5 的本地回归测试：词法、语法、语义、IRSim、SPIM 和优化后 IRSim 验证均可执行。后续如果继续完善，可以重点改进寄存器分配、全局数据流优化、目标代码指令选择质量和更完整的异常输入处理。